

FINDING THE k SHORTEST PATHS*

DAVID EPPSTEIN†

Abstract. We give algorithms for finding the k shortest paths (not required to be simple) connecting a pair of vertices in a digraph. Our algorithms output an implicit representation of these paths in a digraph with n vertices and m edges, in time $O(m + n \log n + k)$. We can also find the k shortest paths from a given source s to each vertex in the graph, in total time $O(m + n \log n + kn)$. We describe applications to dynamic programming problems including the knapsack problem, sequence alignment, maximum inscribed polygons, and genealogical relationship discovery.

Key words. shortest paths, network programming, path enumeration, near-optimal solutions, dynamic programming, knapsack problem, sequence alignment, inscribed polygon, genealogy

AMS subject classifications. 05C12, 05C85, 94C15

PII. S0097539795290477

1. Introduction. We consider a long-studied generalization of the shortest path problem, in which not one but several short paths must be produced. The *k -shortest-paths problem* is to list the k paths connecting a given source-destination pair in the digraph with minimum total length. Our techniques also apply to the problem of listing all paths shorter than some given threshold length. In the version of these problems studied here, cycles of repeated vertices are allowed. We first present a basic version of our algorithm, which is simple enough to be suitable for practical implementation while losing only a logarithmic factor in time complexity. We then show how to achieve optimal time (constant time per path once a shortest path tree has been computed) by applying Frederickson's [26] algorithm for finding the minimum k elements in a *heap-ordered tree*.

1.1. Applications. The applications of shortest path computations are too numerous to cite in detail. They include situations in which an actual path is the desired output, such as robot motion planning, highway and power line engineering, and network connection routing. They include problems of scheduling such as critical path computation in PERT charts. Many optimization problems solved by dynamic programming or more complicated matrix searching techniques, such as the knapsack problem, sequence alignment in molecular biology, construction of optimal inscribed polygons, and length-limited Huffman coding, can be expressed as shortest path problems.

Methods for finding k shortest paths have been applied to many of these applications, for several reasons.

- *Additional constraints.* One may wish to find a path that satisfies certain constraints beyond having a small length, but those other constraints may be ill defined or hard to optimize. For instance, in power transmission route selection [18], a power line should connect its endpoints reasonably directly, but there may be more or less community support for one option or another. A typical solution is to compute several short paths and then choose among

*Received by the editors August 18, 1995; accepted for publication (in revised form) April 17, 1997; published electronically August 4, 1998. This research was supported in part by NSF grant CCR-9258355 and by matching funds from Xerox Corporation.

<http://www.siam.org/journals/sicomp/28-2/29047.html>

†Department of Information and Computer Science, University of California, Irvine, CA 92697-3425 (eppstein@ics.uci.edu, <http://www.ics.uci.edu/~eppstein>).

them by considering the other criteria. We recently implemented a similar technique as a heuristic for the NP-hard problem of, given a graph with colored edges, finding a shortest path using each color at most once [20]. This type of application is the main motivation cited by Dreyfus [17] and Lawler [39] for k -shortest-path computations.

- *Model evaluation.* Paths may be used to model problems that have known solutions, independent of the path formulation; for instance, in a k -shortest-path model of automatic translation between natural languages [30], a correct translation can be found by a human expert. By listing paths until this known solution appears, one can determine how well the model fits the problem, in terms of the number of incorrect paths seen before the correct path. This information can be used to tune the model as well as to determine the number of paths that need to be generated when applying additional constraints to search for the correct solution.
- *Sensitivity analysis.* By computing more than one shortest path, one can determine how sensitive the optimal solution is to variation of the problem's parameters. In biological sequence alignment, for example, one typically wishes to see several "good" alignments rather than one optimal alignment; by comparing these several alignments, biologists can determine which portions of an alignment are most essential [8, 64]. This problem can be reduced to finding several shortest paths in a grid graph.
- *Generation of alternatives.* It may be useful to examine not just the optimal solution to a problem but a larger class of solutions, to gain a better understanding of the problem. For example, the states of a complex system might be represented as a finite state machine, essentially just a graph, with different probabilities on each state transition edge. In such a model, one would likely want to know not just the chain of events most likely to lead to a failure state but rather all chains having a failure probability over some threshold. Taking the logarithms of the transition probabilities transforms this problem into one of finding all paths shorter than a given length.

We later discuss in more detail some of the dynamic programming applications listed above and show how to find the k best solutions to these problems by using our shortest path algorithms. As well as improving previous solutions to the general k -shortest-paths problem, our results improve more specialized algorithms for finding length-bounded paths in the grid graphs arising in sequence alignment [8] and for finding the k best solutions to the knapsack problem [15].

1.2. New results. We prove the following results. In all cases we assume we are given a digraph in which each edge has a nonnegative length. We allow the digraph to contain self-loops and multiple edges. In each case the paths are output in an implicit representation from which simple properties such as the length are available in constant time per path. We may explicitly list the edges in any path in time proportional to the number of edges.

- We find the k shortest paths (allowing cycles) connecting a given pair of vertices in a digraph in time $O(m + n \log n + k)$.
- We find the k shortest paths from a given source in a digraph to each other vertex in time $O(m + n \log n + kn)$.

We can also solve the similar problem of finding all paths shorter than a given length, with the same time bounds. The same techniques apply to digraphs with negative edge lengths but no negative cycles, but the time bounds above should be

modified to include the time to compute a single source shortest path tree in such networks, $O(mn)$ [6, 23] or $O(mn^{1/2} \log N)$ where all edge lengths are integers and N is the absolute value of the most negative edge length [29]. For a directed acyclic graph (DAG), with or without negative edge lengths, shortest path trees can be constructed in linear time and the $O(n \log n)$ term above can be omitted. The related problem of finding the k longest paths in a DAG [4] can be transformed to a shortest path problem simply by negating all edge lengths; we can therefore also solve it in the same time bounds.

1.3. Related work. Many papers study algorithms for k shortest paths [3, 5, 7, 9, 13, 14, 17, 24, 31, 32, 34, 35, 37, 38, 39, 40, 41, 43, 44, 45, 47, 50, 51, 56, 57, 58, 59, 60, 63, 65, 66, 67, 68, 69]. Dreyfus [17] and Yen [69] cite several additional papers on the subject going back as far as 1957.

One must distinguish several common variations of the problem. In many of the papers cited above, the paths are restricted to be *simple*; i.e., no vertex can be repeated. This has advantages in some applications, but as our results show this restriction seems to make the problem significantly harder. Several papers [3, 13, 17, 24, 41, 42, 58, 59] consider the version of the k -shortest-paths problem in which repeated vertices are allowed, and it is this version that we also study. Of course, for the DAGs that arise in many of the applications described above, including scheduling and dynamic programming, no path can have a repeated vertex and the two versions of the problem become equivalent. Note also that in the application described earlier of listing the most likely failure paths of a system modelled by a finite state machine, it is the version studied here rather than the more common simple path version that one wants to solve.

One can also make a restriction that the paths found be edge disjoint or vertex disjoint [61] or include capacities on the edges [10, 11, 12, 49]; however, such changes turn the problem into one more closely related to network flow.

Fox [24] gives a method for the k -shortest-path problem based on Dijkstra's algorithm, which with more recent improvements in priority queue data structures [27] takes time $O(m + kn \log n)$; this seems to be the best previously known k -shortest-paths algorithm. Dreyfus [17] mentions the version of the problem in which we must find paths from one source to each other vertex in the graph, and describes a simple $O(kn^2)$ time dynamic programming solution to this problem. For the k -shortest-simple-paths problem, the best known bound is $O(k(m + n \log n))$ in undirected graphs [35] or $O(kn(m + n \log n))$ in directed graphs ([39], again including more recent improvements in Dijkstra's algorithm). Thus all previous algorithms took time $O(n \log n)$ or more per path. We improve this to constant time per path.

A similar problem to the one studied here is that of finding the k minimum weight spanning trees in a graph. Recent algorithms for this problem [22, 21, 25] reduce it to finding the k minimum weight nodes in a *heap-ordered tree*, defined using the *best swap* in a sequence of graphs. Heap-ordered tree selection has also been used to find the smallest interpoint distances or the nearest neighbors in geometric point sets [16]. We apply a similar tree selection technique to the k -shortest-path problem; however, the reduction of k shortest paths to heap-ordered trees is very different from the constructions in these other problems.

2. The basic algorithm. Finding the k shortest paths between two terminals s and t has been a difficult enough problem to warrant much research. In contrast, the similar problem of finding paths with only one terminal s , ending anywhere in the graph, is much easier: one can simply use breadth first search. Maintain a priority

queue of paths, initially containing the single zero-edge path from s to itself; then repeatedly remove the shortest path from the priority queue, add it to the list of output paths, and add all one-edge extensions of that path to the priority queue. If the graph has bounded degree d , a breadth first search from s until k paths are found takes time $O(dk + k \log k)$; note that this bound does not depend in any way on the overall size of the graph. If the paths need not be output in order by length, Frederickson's heap selection algorithm [26] can be used to speed this up to $O(dk)$.

The main idea of our k -shortest-paths algorithm, then, is to translate the problem from one with two terminals, s and t , to a problem with only one terminal. One can find paths from s to t simply by finding paths from s to any other vertex and concatenating a shortest path from that vertex to t . However, we cannot simply apply this idea directly, for several reasons: (1) There is no obvious relation between the ordering of the paths from s to other vertices and the ordering of the corresponding paths from s to t . (2) Each path from s to t may be represented in many ways as a path from s to some vertex followed by a shortest path from that vertex to t . (3) Our input graph may not have bounded degree.

In outline, we deal with problem (1) by using a potential function to modify the edge lengths in the graph so that the length of any shortest path to t is zero; therefore concatenating such paths to paths from s will preserve the ordering of the path lengths. We deal with problem (2) by considering only paths from s in which the last edge is not in a fixed shortest path tree to t ; this leads to the implicit representation we use to represent each path in constant space. (Ideas similar to these appear also in [46].) However, this solution gives rise to a fourth problem: (4) We do not wish to spend much time searching edges of the shortest path tree, as this time can not be charged against newly found s - t paths.

The heart of our algorithm is the solution to problems (3) and (4). Our idea is to construct a binary heap for each vertex, listing the edges that are not part of the shortest path tree and that can be reached from that vertex by shortest-path-tree edges. In order to save time and space, we use persistence techniques to allow these heaps to share common structures with each other. In the basic version of the algorithm, this collection of heaps forms a bounded-degree graph having $O(m+n \log n)$ vertices. Later we show how to improve the time and space bounds of this part of the algorithm using tree decomposition techniques of Frederickson [25].

2.1. Preliminaries. We assume throughout that our input graph G has n vertices and m edges. We allow self-loops and multiple edges, so m may be larger than $\binom{n}{2}$. The *length* of an edge e is denoted $\ell(e)$. By extension we can define the length $\ell(p)$ for any path in G to be the sum of its edge lengths. The *distance* $d(s, t)$ for a given pair of vertices is the length of the shortest path starting at s and ending at t ; with the assumption of no negative cycles, this is well defined. Note that $d(s, t)$ may be unequal to $d(t, s)$. The two endpoints of a directed edge e are denoted $\text{tail}(e)$ and $\text{head}(e)$; the edge is directed from $\text{tail}(e)$ to $\text{head}(e)$.

For our purposes, a *heap* is a binary tree in which vertices have weights, satisfying the restriction that the weight of any vertex is less than or equal to the minimum weight of its children. We will not always care whether the tree is balanced (and in some circumstances we will allow trees with infinite depth). More generally, a D -*heap* is a degree- D tree with the same weight-ordering property; thus the usual heaps above are 2-heaps. As is well known (e.g., see [62]), any set of values can be placed into a balanced heap by the *heapify* operation in linear time. In a balanced heap, any new element can be inserted in logarithmic time. We can list the elements of a heap in

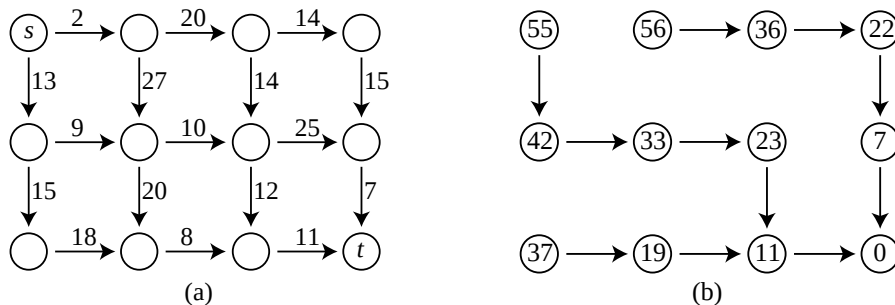


FIG. 1. (a) Example digraph G with edge lengths and specified terminals; (b) shortest path tree T and distances to t in G .

order by weight, taking logarithmic time to generate each element, simply by using breadth first search.

2.2. Implicit representation of paths. As discussed earlier, our algorithm does not output each path it finds explicitly as a sequence of edges; instead it uses an implicit representation, described in this section.

The i th shortest path in a digraph may have $\Omega(ni)$ edges, so the best time we could hope for in an explicit listing of shortest paths would be $O(k^2n)$. Our time bounds are faster than this, so we must use an implicit representation for the paths. However, our representation is not a serious obstacle to use of our algorithm: we can list the edges of any path we output in time proportional to the number of edges, and simple properties (such as the length) are available in constant time. Similar implicit representations have previously been used for related problems such as the k minimum weight spanning trees [22, 21, 25]. Further, previous papers on the k -shortest-path problem give time bounds omitting the $O(k^2n)$ term above, so these papers must tacitly or not be using an implicit representation.

Our representation is similar in spirit to those used for the k minimum weight spanning trees problem: for that problem, each successive tree differs from a previously listed tree by a *swap*, the insertion of one edge and removal of another edge. The implicit representation consists of a pointer to the previous tree and a description of the swap. For the shortest path problem, each successive path will turn out to differ from a previously listed path by the inclusion of a single edge not part of a shortest path tree and appropriate adjustments in the portion of the path that involves shortest path tree edges. Our implicit representation consists of a pointer to the previous path, and a description of the newly added edge.

Given s and t in a digraph G (Figure 1(a)), let T be a single-destination shortest path tree with t as destination (Figure 1(b)); this is the same as a single source shortest path tree in the graph G^R formed by reversing each edge of G . We can compute T in time $O(m + n \log n)$ [27]. We denote by $next_T(v)$ the next vertex reached after v on the path from v to t in T .

Given an edge e in G , define

$$\delta(e) = \ell(e) + d(head(e), t) - d(tail(e), t).$$

Intuitively, $\delta(e)$ measures how much distance is lost by being “sidetracked” along e instead of taking a shortest path to t . The values of δ for our example graph are shown in Figure 2(a).

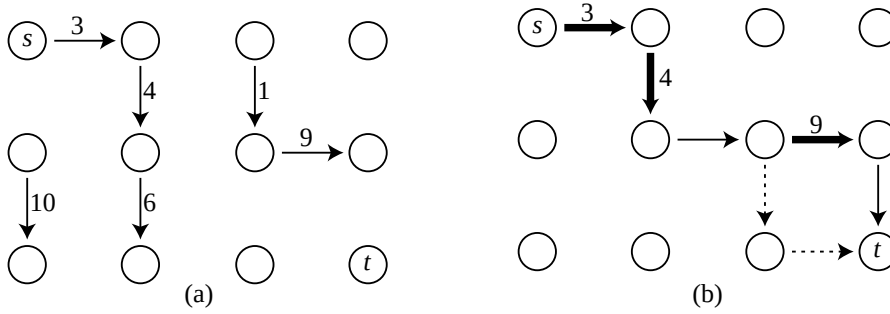


FIG. 2. (a) Edges in $G - T$ labeled by $\delta(e)$ ($\delta(e) = 0$ for edges in T); (b) path p , $\text{sidetracks}(p)$ (the heavy edges, labeled 3, 4, and 9), and $\text{prefix}(p)$ (differing from p in the two dashed edges; $\text{sidetracks}(\text{prefix}(p))$ consists of the two edges labeled 3 and 4).

LEMMA 1. For any $e \in G$, $\delta(e) \geq 0$. For any $e \in T$, $\delta(e) = 0$.

For any path p in G , formed by a sequence of edges, some edges of p may be in T , and some others may be in $G - T$. Any path p from s to t is uniquely determined solely by the subsequence $\text{sidetracks}(p)$ of its edges in $G - T$ (Figure 2(b)). For, given a pair of edges in the subsequence, there is a uniquely determined way of inserting edges from T so that the head of the first edge is connected to the tail of the second edge. As an example, the shortest path in T from s to t is represented by the empty sequence. A sequence of edges in $G - T$ may not correspond to any s - t path, if it includes a pair of edges that cannot be connected by a path in T . If $S = \text{sidetracks}(p)$, we define $\text{path}(S)$ to be the path p .

Our implicit representation will involve these sequences of edges in $G - T$. We next show how to recover $\ell(p)$ from information in $\text{sidetracks}(p)$.

For any nonempty sequence S of edges in $G - T$, let $\text{prefix}(S)$ be the sequence formed by the removal of the last edge in S . If $S = \text{sidetracks}(p)$, then we denote this last sidetrack edge by $\text{lastsidetrack}(p)$; $\text{prefix}(S)$ will define a path $\text{prefix}(p) = \text{path}(\text{prefix}(S))$ (Figure 2(b)).

LEMMA 2. For any path p from s to t ,

$$\ell(p) = d(s, t) + \sum_{e \in \text{sidetracks}(p)} \delta(e) = d(s, t) + \sum_{e \in p} \delta(e).$$

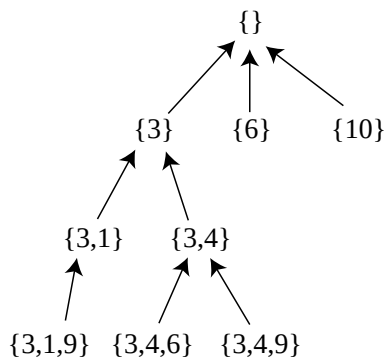
LEMMA 3. For any path p from s to t in G , for which $\text{sidetracks}(p)$ is nonempty, $\ell(p) \geq \ell(\text{prefix}(p))$.

Our representation of a path p in the list of paths produced by our algorithm will then consist of two components:

- the position in the list of $\text{prefix}(p)$.
- edge $\text{lastsidetrack}(p)$.

Although the final version of our algorithm, which uses Frederickson's heap selection technique, does not necessarily output paths in sorted order, we will nevertheless be able to guarantee that $\text{prefix}(p)$ is output before p . One can easily recover p itself from our representation in time proportional to the number of edges in p . The length $\ell(p)$ for each path can easily be computed as $\delta(\text{lastsidetrack}(p)) + \ell(\text{prefix}(p))$. We will see later that we can also compute many other simple properties of the paths in constant time per path.

2.3. Representing paths by a heap. The representation of s - t paths discussed in the previous section gives a natural tree of paths, in which the parent of any path p

FIG. 3. Tree of paths, labeled by $\text{sidetracks}(p)$.

is $\text{prefpath}(p)$ (Figure 3). The degree of any node in this *path tree* is at most m , since there can be at most one child for each possible value of $\text{lastsidetrack}(p)$. The possible values of $\text{lastsidetrack}(q)$ for paths q that are children of p are exactly those edges in $G - T$ that have tails on the path from $\text{head}(\text{lastsidetrack}(p))$ to t in the shortest path tree T .

If G contains cycles, the path tree is infinite. By Lemma 3, the path tree is heap-ordered. However, since its degree is not necessarily constant, we cannot directly apply breadth first search (nor Frederickson's heap selection technique, described later in Lemma 8) to find its k minimum values. Instead we form a heap by replacing each node p of the path tree with an equivalent bounded-degree subtree (essentially, a heap of the edges with tails on the path from $\text{head}(\text{lastsidetrack}(p))$ to t , ordered by $\delta(e)$). We must also take care that we do this in such a way that the portion of the path tree explored by our algorithm can be easily constructed.

For each vertex v we wish to form a heap $H_G(v)$ for all edges with tails on the path from v to t , ordered by $\delta(e)$. We will later use this heap to modify the path tree by replacing each node p with a copy of $H_G(\text{head}(\text{lastsidetrack}(p)))$.

Let $\text{out}(v)$ denote the edges in $G - T$ with tails at v (Figure 4(a)). We first build a heap $H_{\text{out}}(v)$ for each vertex v of the edges in $\text{out}(v)$ (Figure 4(b)). The weights used for the heap are simply the values $\delta(e)$ defined earlier. $H_{\text{out}}(v)$ will be a 2-heap with the added restriction that the root of the heap only has one child. It can be built for each v in time $O(|\text{out}(v)|)$ by letting the root $\text{outroot}(v)$ be the edge minimizing $\delta(e)$ in $\text{out}(v)$ and letting its child be a heap formed by heapification of the rest of the edges in $\text{out}(v)$. The total time for this process is $\sum O(|\text{out}(v)|) = O(m)$.

We next form the heap $H_G(v)$ by merging all heaps $H_{\text{out}}(w)$ for w on the path in T from v to t . More specifically, for each vertex v we merge $H_{\text{out}}(v)$ into $H_G(\text{next}_T(v))$ to form $H_G(v)$. We will continue to need $H_G(\text{next}_T(v))$, so this merger should be done in a persistent (nondestructive) fashion.

We guide this merger of heaps using a balanced heap $H_T(v)$ for each vertex v , containing only the roots $\text{outroot}(w)$ of the heaps $H_{\text{out}}(w)$, for each w on the path from v to t . $H_T(v)$ is formed by inserting $\text{outroot}(v)$ into $H_T(\text{next}_T(v))$ (Figure 5(a)). To perform this insertion persistently, we create new copies of the nodes on the path updated by the insertion (marked by asterisks in Figure 5(a)), with appropriate pointers to the other, unchanged members of $H_T(\text{next}_T(v))$. Thus we can store $H_T(v)$ without changing $H_T(\text{next}_T(v))$ by using an additional $O(\log n)$ words of memory to store only the nodes on that path.

We now form $H_G(v)$ by connecting each node $\text{outroot}(w)$ in $H_T(v)$ to an additional

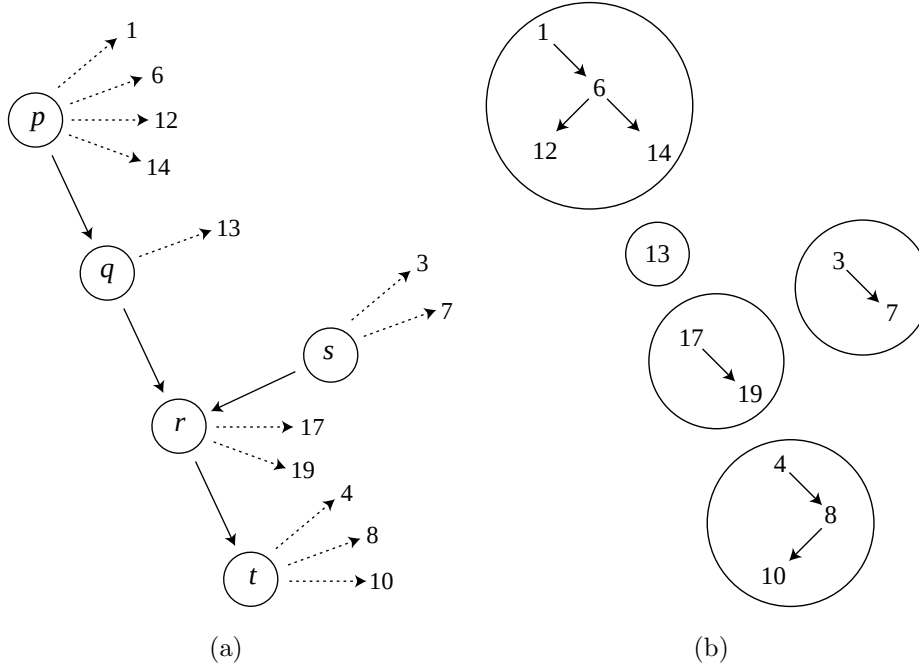


FIG. 4. (a) Portion of a shortest path tree, showing $out(v)$ and corresponding values of δ ; (b) $H_{out}(v)$.

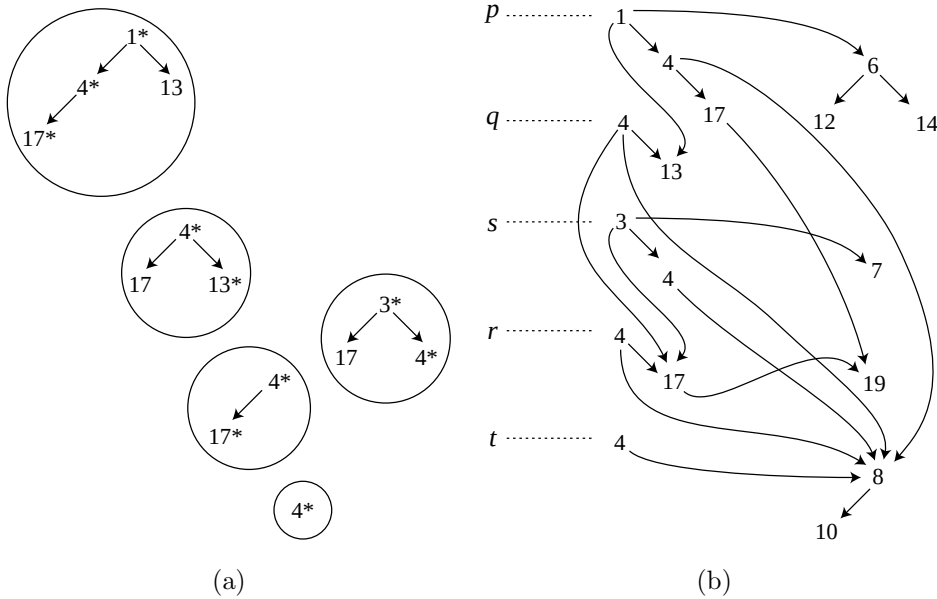


FIG. 5. (a) $H_T(v)$ with asterisks marking path of nodes updated by insertion of $outroot(v)$ into $H_T(next_T(v))$; (b) $D(G)$ has a node for each marked node in Figure 5(a) and each nonroot node in Figure 4(b).

subtree beyond the two it points to in $H_T(v)$, namely, to the rest of heap $H_{out}(w)$.

$H_G(v)$ can be constructed at the same time as we construct $H_T(v)$, with a similar amount of work. $H_G(v)$ is thus a 3-heap, as each node includes at most three children,

either two from $H_T(v)$ and one from $H_{out}(w)$ or none from $H_T(v)$ and two from $H_{out}(w)$.

We summarize the construction so far in a form that emphasizes the shared structure in the various heaps $H_G(v)$.

LEMMA 4. *In time $O(m + n \log n)$ we can construct a DAG $D(G)$ and a map from vertices $v \in G$ to $h(v) \in D(G)$, with the following properties:*

- $D(G)$ has $O(m + n \log n)$ vertices.
- Each vertex in $D(G)$ corresponds to an edge in $G - T$.
- Each vertex in $D(G)$ has out-degree at most 3.
- The vertices reachable in $D(G)$ from $h(v)$ form a 3-heap $H_G(v)$ in which the vertices of the heap correspond to edges of $G - T$ with tails on the path in T from v to t , in heap order by the values of $\delta(e)$.

Proof. The vertices in $D(G)$ come from two sources: heaps $H_{out}(v)$ and $H_T(v)$. Each node in $H_{out}(v)$ corresponds to a unique edge in $G - T$, so there are at most $m - n + 1$ nodes coming from heaps $H_{out}(v)$. Each vertex of G also contributes $\lfloor \log_2 i \rfloor$ nodes from heaps $H_T(v)$, where i is the length of the path from the vertex to t , $1 + \lfloor \log_2 i \rfloor$ measures the number of balanced binary heap nodes that need to be updated when inserting $outroot(v)$ into $H_T(next_T(v))$, and we subtract one because $outroot(v)$ itself was already included in our total for $H_{out}(v)$. In the worst case, T is a path and the total contribution is at most $\sum_i \lfloor \log_2 i \rfloor \leq n \log_2 n - cn$, where c varies between roughly 1.91 and 2 depending on the ratio of n to the nearest power of two. Therefore the total number of nodes in $D(G)$ is at most $m + n \log_2 n - (c + 1)n$. The degree bound follows from the construction, and it is straightforward to construct $D(G)$ as described above in constant time per node, after computing the shortest path tree T in time $O(m + n \log n)$ using Fibonacci heaps [27].

Map $h(v)$ simply takes v to the root of $H_G(v)$. For any vertex v in $D(G)$, let $\delta(v)$ be a shorthand for $\delta(e)$, where e is the edge in G corresponding to v . By construction, the nodes reachable from $h(v)$ are those in $H_T(v)$ together with, for each such node w , the rest of the nodes in $H_{out}(w)$; $H_T(v)$ was constructed to correspond exactly to the vertices on the path from v to t , and $H_{out}(w)$ represents the edges with tails at each vertex, so together these reachable nodes represent all edges with tails on the path. Each edge (u, v) in $D(G)$ corresponds to an edge either in some $H_T(w)$ or in some $H_{out}(w)$, and in either case the heap ordering for $D(G)$ is a consequence of the ordering in these smaller heaps. $\square$

$D(G)$ is shown in Figure 5(b). The nodes reachable from s in $D(G)$ form a structure $H_G(s)$ representing the paths differing from the original shortest path by the addition of a single edge in $G - T$. We now describe how to augment $D(G)$ with additional edges to produce a graph which can represent all s - t paths, not just those paths with a single edge in $G - T$.

We define the *path graph* $P(G)$ as follows. The vertices of $P(G)$ are those of $D(G)$, with one additional vertex, the *root* $r = r(s)$. The vertices of $P(G)$ are unweighted, but the edges are given lengths. For each directed edge (u, v) in $D(G)$, we create the edge between the corresponding vertices in $P(G)$, with length $\delta(v) - \delta(u)$. We call such edges *heap edges*. For each vertex v in $P(G)$, corresponding to an edge in $G - T$ connecting some pair of vertices u and w , we create a new edge from v to $h(w)$ in $P(G)$, having as its length $\delta(h(w))$. We call such edges *cross edges*. We also create

an *initial edge* between r and $h(s)$, having as its length $\delta(h(s))$.

$P(G)$ has $O(m + n \log n)$ vertices, each with out-degree at most four. It can be constructed in time $O(m + n \log n)$.

LEMMA 5. *There is a one-to-one length-preserving correspondence between s - t paths in G , and paths starting from r in $P(G)$.*

Proof. Recall that an s - t path p in G is uniquely defined by $\text{sidetracks}(p)$, the sequence of edges from p in $G - T$. We now show that for any such sequence, there corresponds a unique path from r in $P(G)$ ending at a node corresponding to $\text{lastsidetrack}(p)$, and conversely any path from r in $P(G)$ corresponds to $\text{sidetracks}(p)$ for some path p .

Given a path p in G , we construct a corresponding path p' in $P(G)$ as follows. If $\text{sidetracks}(p)$ is empty (i.e., p is the shortest path), we let p' consist of the single node r . Otherwise, form a path q' in $P(G)$ corresponding to $\text{prefpath}(p)$ by induction on the length of $\text{sidetracks}(p)$. By induction, q' ends at a node of $P(G)$ corresponding to edge $(u, v) = \text{lastsidetrack}(\text{prefpath}(p))$. When we formed $P(G)$ from $D(G)$, we added an edge from this node to $h(v)$. Since $\text{lastsidetrack}(p)$ has its tail on the path in T from v to t , it corresponds to a unique node in $H_G(v)$, and we form p' by concatenating q' with the path from $h(v)$ to that node. The edge lengths on this concatenated path telescope to $\delta(\text{lastsidetrack}(p))$, and $\ell(p) = \ell(\text{prefpath}(p)) + \ell(\text{lastsidetrack}(p))$ by Lemma 2, so by induction $\ell(p) = \ell(q') + \ell(\text{lastsidetrack}(p)) = \ell(p')$.

Conversely, to construct an s - t path in G from a path p' in $P(G)$, we construct a sequence of edges in G , $\text{pathseq}(p')$. If p' is empty, $\text{pathseq}(p')$ is also empty. Otherwise $\text{pathseq}(p')$ is formed by taking in sequence the edges in G corresponding to tails of cross edges in p' and adding at the end of the sequence the edge in G corresponding to the final vertex of p' . Since the nodes of $P(G)$ reachable from the head of each cross edge (u, v) are exactly those in $H_G(v)$, each successive edge added to $\text{pathseq}(p')$ is on the path in T from v to t , and $\text{pathseq}(p')$ is of the form $\text{sidetracks}(p)$ for some path p in G . $\square$

LEMMA 6. *In $O(m + n \log n)$ time we can construct a graph $P(G)$ with a distinguished vertex r , having the following properties.*

- $P(G)$ has $O(m + n \log n)$ vertices.
- Each vertex of $P(G)$ has out-degree at most four.
- Each edge of $P(G)$ has nonnegative weight.
- There is a one-to-one correspondence between s - t paths in G and paths starting from r in $P(G)$.
- The correspondence preserves lengths of paths in that length ℓ in $P(G)$ corresponds to length $d(s, t) + \ell$ in G .

Proof. The bounds on size, time, and out-degree follow from Lemma 4, and the nonnegativity of edge weights follows from the heap ordering proven in that lemma. The correctness of the correspondence between paths in G and in $P(G)$ is shown above in Lemma 5. $\square$

To complete our construction, we find from the path graph $P(G)$ a 4-heap $H(G)$, so that the nodes in $H(G)$ represent paths in G . $H(G)$ is constructed by forming a node for each path in $P(G)$ rooted at r . The parent of a node is the path with one fewer edge. Since $P(G)$ has out-degree four, each node has at most four children. Weights are heap-ordered, and the weight of a node is the length of the corresponding path.

LEMMA 7. *$H(G)$ is a 4-heap in which there is a one-to-one correspondence between nodes and s - t paths in G , and in which the length of a path in G is $d(s, t)$*

plus the weight of the corresponding node in $H(G)$.

We note that, if an algorithm explores a connected region of $O(k)$ nodes in $H(G)$, it can represent the nodes in constant space by assigning them numbers and indicating for each node its parent and the additional edge in the corresponding path of $P(G)$. The children of a node are easy to find simply by following appropriate out-edges in $P(G)$, and the weight of a node is easy to compute from the weight of its parent. It is also easy to maintain along with this representation the corresponding implicit representation of s - t paths in G .

2.4. Finding the k shortest paths.

THEOREM 1. *In time $O(m + n \log n)$ we can construct a data structure that will output the shortest paths from s to t in a graph in order by weight, taking time $O(\log i)$ to output the i th path.*

Proof. We apply breadth first search to $P(G)$, as described at the start of the section, and translate the search results to paths using the correspondence described above. $\square$

We next describe how to compute paths from s to all n vertices of the graph. In fact our construction solves more easily the reverse problem of finding paths from each vertex to the destination t . The construction of $P(G)$ is as above, except that instead of adding a single root $r(s)$ connected to $h(s)$, we add a root $r(v)$ for each vertex $v \in G$. The modification to $P(G)$ takes $O(n)$ time. Using the modified $P(G)$, we can compute a heap $H_v(G)$ of paths from each v to t and compute the k smallest such paths in time $O(k)$.

THEOREM 2. *Given a source vertex s in a digraph G , we can find in time $O(m + n \log n + kn \log k)$ an implicit representation of the k shortest paths from s to each other vertex in G .*

Proof. We apply the construction above to G^R , with s as destination. We form the modified path graph $P(G^R)$, find for each vertex v a heap $H_v(G^R)$ of paths in G^R from v to s , and apply breadth first search to this heap. Each resulting path corresponds to a path from s to v in G . $\square$

3. Improved space and time. The basic algorithm described above takes time $O(m + n \log n + k \log k)$, even if a shortest path tree has been given. If the graph is sparse, the $n \log n$ term makes this bound nonlinear. This term comes from two parts of our method, Dijkstra's shortest path algorithm and the construction of $P(G)$ from the tree of shortest paths. But for certain graphs, or with certain assumptions about edge lengths, shortest paths can be computed more quickly than $O(m + n \log n)$ [2, 28, 33, 36], and in these cases we would like to speed up our construction of $P(G)$ to match these improvements. In other cases, k may be large and the $k \log k$ term may dominate the time bound; again we would like to improve this nonlinear term. In this section we show how to reduce the time for our algorithm to $O(m + n + k)$, assuming a shortest path tree is given in the input. As a consequence we can also improve the space used by our algorithm.

3.1. Faster heap selection. The following result is due to Frederickson [26].

LEMMA 8. *We can find the k smallest weight vertices in any heap in time $O(k)$.*

Frederickson's result applies directly to 2-heaps, but we can easily extend it to D -heaps for any constant D . One simple method of doing this involves forming a 2-heap from the given D -heap by making $D - 1$ copies of each vertex, connected in a binary tree with the D children as leaves, and breaking ties in such a way that the Dk smallest weight vertices in the 2-heap correspond exactly to the k smallest weights in

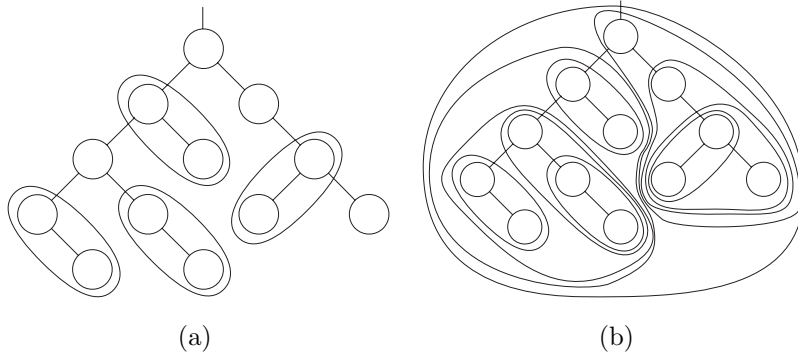


FIG. 6. (a) *Restricted partition of order 2*; (b) *multilevel partition*.

the D -heap.

By using this algorithm in place of breadth first search, we can reduce the $O(k \log k)$ term in our time bounds to $O(k)$.

3.2. Faster path heap construction. Recall that the bottleneck of our algorithm is the construction of $H_T(v)$, a heap for each vertex v in G of those vertices on the path from v to t in the shortest path tree T . The vertices in $H_T(v)$ are in heap order by $\delta(\text{outroot}(u))$. In this section we consider the abstract problem, given a tree T with weighted nodes, of constructing a heap $H_T(v)$ for each vertex v of the other nodes on the path from v to the root of the tree. The construction of Lemma 4 solves this problem in time and space $O(n \log n)$; here we give a more efficient but also more complicated solution.

By introducing dummy nodes with large weights, we can assume without loss of generality that T is binary and that the root t of T has indegree one. We will also assume that all vertex weights in T are distinct; this can be achieved at no loss in asymptotic complexity by use of a suitable tie-breaking rule. We use the following technique of Frederickson [25].

DEFINITION 1. *A restricted partition of order z with respect to a rooted binary tree T is a partition of the vertices of V such that*

1. *each set in the partition contains at most z vertices;*
2. *each set in the partition induces a connected subtree of T ;*
3. *for each set S in the partition, if S contains more than one vertex, then there are at most two tree edges having one endpoint in S ;*
4. *no two sets can be combined and still satisfy the other conditions.*

In general such a partition can easily be found in linear time by merging sets until we get stuck. However for our application, z will always be 2 (Figure 6(a)), and by working from the bottom up we can find an optimal partition in linear time.

LEMMA 9 (Frederickson [25]). *In linear time we can find an order-2 partition of a binary tree T for which there are at most $5n/6$ sets in the partition.*

Contracting each set in a restricted partition gives again a binary tree. We form a *multilevel* partition [25] by recursively partitioning this contracted binary tree (Figure 6(b)). We define a sequence of trees T_i as follows. Let $T_0 = T$. For any $i > 0$, let T_i be formed from T_{i-1} by performing a restricted partition as above and contracting the resulting sets. Then $|T_i| = O((5/6)^i n)$.

For any set S of vertices in T_{i-1} contracted to form a vertex v in T_i , define $\text{nextlevel}(S)$ to be the set in the partition of T_i containing S . We say that S is an

interior set if it is contracted to a degree two vertex. Note that if t has indegree one, the same is true for the root of any T_i , so t is not part of any interior set, and each interior set has one incoming and one outgoing edge. Since T_i is a contraction of T , each edge in T_i corresponds to an edge in T . Let e be the outgoing edge from v in T_i ; then we define $rootpath(S)$ to be the path in T from $head(e)$ to t . If S is an interior set, with a single incoming edge e' , we let $inpath(S)$ be the path in T from $head(e')$ to $tail(e)$.

Define an m -partial heap to be a pair (M, H) , where H is a heap and M is a set of m elements each smaller than all nodes in H . If H is empty, M can have fewer than m elements and we will still call (M, H) an m -partial heap.

Let us outline the structures used in our algorithm, before describing the details of computing these structures. We first find a partial heap $(M_1(S), H_1(S))$ for the vertices of T in each path $inpath(S)$. Although our algorithm performs an interleaved construction of all of these sets at once, it is easiest to define them from the top down by defining $M_1(S)$ for a set S in the partition of T_{i-1} in terms of similar sets in T_i and higher levels of the multilevel partition. Specifically, let $M_2(S)$ denote those elements in $M_1(S')$ for those S' containing S at higher levels of the multilevel partition, and let $k = \max(i+2, |M_2(S)|+1)$; then we define $M_1(S)$ to be the vertices in $inpath(S)$ having the k smallest vertex weights. Our algorithm for computing $H_1(S)$ from the remaining vertices on $inpath(S)$ involves an intermediate heap $H_2(S')$ formed by adding the vertices in $M_1(S') - M_1(S)$ to $H_1(S')$, where S' consists of one or both of the subsets of S contracted at the next lower level of the decomposition and containing vertices of $inpath(S)$. After a bottom-up computation of M_1 , H_1 , and H_2 , we then perform a top-down computation of a family of $(i+1)$ -partial heaps, $(M_3(S), H_3(S))$; M_3 is formed by removing some elements from M_1 and H_3 is formed by adding those elements to H_1 . Finally, the desired output $H_T(v)$ can be constructed from the 1-partial heap $(M_3(v), H_3(v))$ at level T_0 in the decomposition.

Before describing our algorithms, let us bound a quantity useful in their analysis. Let m_i denote the sum of $|M_1(S)|$ over sets S contracted in T_i .

LEMMA 10. *For each i , $m_i = O(i|T_i|)$.*

Proof. By the definition of $M_1(S)$ above,

$$m_i = \sum_S \max(i+2, |M_2(S)|+1) \leq \sum_S |M_2(S)| + i+2 \leq (i+2)|T_i| + \sum_S |M_2(S)|.$$

All sets $M_2(S)$ appearing in this sum are disjoint, and all are included in m_{i+1} , so we can simplify this formula to

$$m_i \leq (i+2)|T_i| + m_{i+1} \leq \sum_{j \geq i} (j+2)|T_j| \leq \sum_{j \geq i} (j+2) \left(\frac{5}{6}\right)^{j-i} |T_i| = O(i|T_i|). \quad \square$$

We use the following data structure to compute the sets $M_1(S)$ (which, recall, are sets of low-weight vertices on $inpath(S)$). For each interior set S , we form a priority queue $Q(S)$, from which we can retrieve the smallest weight vertex on $inpath(S)$ not yet in $M_1(S)$. This data structure is very simple: if only one of the two subsets forming S contains vertices on $inpath(S)$, we simply copy the minimum-weight vertex on that subset's priority queue, and otherwise we compare the minimum-weight vertices in each subset's priority queue and select the smaller of the two weights. If one of the two subsets' priority queue values change, this structure can be updated simply by repeating this comparison.

We start by setting all the sets $M_1(S)$ to be empty, then progress from the top down through the multilevel decomposition, testing for each set S in each tree T_i (in decreasing order of i) whether we have already added enough members to $M_1(S)$. If not, we add elements one at a time until there are enough to satisfy the definition above of $|M_1(S)|$. Whenever we add an element to $M_1(S)$ we add the same element to $M_1(S')$ for each lower level subset S' to which it also belongs. An element is added by removing it from $Q(S)$ and from the priority queues of the sets at each lower level. We then update the queues bottom up, recomputing the head of each queue and inserting it in the queue at the next level.

LEMMA 11. *The amount of time to compute $M_1(S)$ for all sets S in the multilevel partition, as described above, is $O(n)$.*

Proof. By Lemma 10, the number of operations in priority queues for subsets of T_i is $O(i|T_i|)$. So the total time is $\sum O(i|T_i|) = O(n \sum i(5/6)^i) = O(n)$. $\square$

We next describe how to compute the heaps $H_1(S)$ for the vertices on $\text{inpath}(S)$ that have not been chosen as part of $M_1(S)$. For this stage we work from the bottom up. Recall that S corresponds to one or two vertices of T_i ; each vertex corresponds to a set S' contracted at a previous level of the multilevel partition. For each such S' along the path in S we will have already formed the partial heap $(M_1(S'), H_1(S'))$. We let $H_2(S')$ be a heap formed by adding the vertices in $M_1(S') - M_1(S)$ to $H_1(S')$. Since $M_1(S') - M_1(S)$ consists of at least one vertex (because of the requirement that $|M_1(S')| \geq |M_1(S)| + 1$), we can form $H_2(S')$ as a 2-heap in which the root has degree one.

If S consists of a single vertex we then let $H_1(S) = H_2(S')$; otherwise we form $H_1(S)$ by combining the two heaps $H_2(S')$ for its two children. The time is constant per set S or linear overall.

We next compute another collection of partial heaps $(M_3(S), H_3(S))$ of vertices in $\text{rootpath}(S)$ for each set S contracted at some level of the tree. If S is a set contracted to a vertex in T_i , we let $(M_3(S), H_3(S))$ be an $(i+1)$ -partial heap. In this phase of the algorithm, we work top down. For each set S , consisting of a collection of vertices in T_{i-1} , we use $(M_3(S), H_3(S))$ to compute for each vertex S' the partial heap $(M_3(S'), H_3(S'))$.

If S consists of a single set S' , or if S' is the parent of the two vertices in S , we let $M_3(S')$ be formed by removing the minimum weight element from $M_3(S)$ and we let $H_3(S')$ be formed by adding that minimum weight element as a new root to $H_3(S)$.

In the remaining case, if S' and $\text{parent}(S')$ are both in S , we form $M_3(S')$ by taking the $i+1$ minimum values in $M_1(\text{parent}(S')) \cup M_3(\text{parent}(S'))$. The remaining values in $M_1(\text{parent}(S')) \cup M_3(\text{parent}(S')) - M_3(S')$ must include at least one value v greater than everything in $H_1(\text{parent}(S'))$. We form $H_3(S')$ by sorting those remaining values into a chain, together with the root of heap $H_3(\text{parent}(S'))$, and connecting v to $H_1(\text{parent}(S'))$.

To complete the process, we compute the heaps $H_T(v)$ for each vertex v . Each such vertex is in T_0 , so the construction above has already produced a 1-partial heap $(M_3(v), H_3(v))$. We must add the value for v itself and produce a true heap, both of which are easy.

LEMMA 12. *Given a tree T with weighted nodes, we can construct for each vertex v a 2-heap $H_T(v)$ of all nodes on the path from v to the root of the tree, in total time and space $O(n)$.*

Proof. The time for constructing (M_1, H_1) has already been analyzed. The only

remaining part of the algorithm that does not take constant time per set is the time for sorting remaining values into a chain, in time $O(i \log i)$ for a set at level i of the construction. The total time at level i is thus $O(|T_i| i \log i)$ which, summed over all i , gives $O(n)$. $\square$

Applying this technique in place of Lemma 4 gives the following result.

THEOREM 3. *Given a digraph G and a shortest path tree from a vertex s , we can find an implicit representation of the k shortest s - t paths in G , in time and space $O(m + n + k)$.*

4. Maintaining path properties. Our algorithm can maintain along with the other information in $H(G)$ various forms of simple information about the corresponding s - t paths in G .

We have already seen that $H(G)$ allows us to recover the lengths of paths. However, lengths are not as difficult as some other information might be to maintain, since they form an additive group. We used this group property in defining $\delta(e)$ to be a difference of path lengths, and in defining edges of $P(G)$ to have weights that were differences of quantities $\delta(e)$.

We now show that we can in fact keep track of any quantity formed by combining information from the edges of the path using any monoid. We assume that there is some given function taking each edge e to an element $value(e)$ of a monoid, and that given two edges e and f we can compute the composite value $value(e) \cdot value(f)$ in constant time. By associativity of monoids, the value $value(p)$ of a path p is well defined. Examples of such values include the path length and number of edges in a path (for which composition is real or integer addition) and the longest or shortest edge in a path (for which composition is minimization or maximization).

Recall that for each vertex we compute a heap $H_G(v)$ representing the sidetracks reachable along the shortest path from v to t . For each node x in $H_G(v)$ we maintain two values: $pathstart(x)$ pointing to a vertex on the path from v to t , and $value(x)$ representing the value of the path from $pathstart(x)$ to the head of the sidetrack edge represented by x . We require that $pathstart$ of the root of the tree is v itself, that $pathstart(x)$ be a vertex between v and the head of the sidetrack edge representing x , and that all descendents of x have $pathstart$ values on the path from $pathstart(x)$ to t . For each edge in $H_G(v)$ connecting nodes x and y we store a further value, representing the value of the path from $pathstart(x)$ to $pathstart(y)$. We also store for each vertex in G the value of the shortest path from v to t .

Then as we compute paths from the root in the heap $H(G)$, representing s - t paths in G , we can keep track of the value of each path merely by composing the stored values of appropriate paths and nodes in the path in $H(G)$. Specifically, when we follow an edge in a heap $H_G(v)$ we include the value stored at that edge, and when we take a sidetrack edge e from a node x in $H_G(v)$ we include $value(x)$ and $value(e)$. Finally we include the value of the shortest path to t from the tail of the last sidetrack edge to t . The portion of the value except for the final shortest path can be updated in constant time from the same information for a shorter path in $H(G)$, and the remaining shortest path value can be included again in constant time, so this computation takes $O(1)$ time per path found.

The remaining difficulty is computing the values $value(x)$, $pathstart(x)$, and also the values of edges in $H_G(v)$.

In the construction of Lemma 4, we need only compute these values for the $O(\log n)$ nodes by which $H_G(v)$ differs from $H_G(parent(v))$, and we can compute each such value as we update the heap in constant time per value. Thus the construction

here goes through with unchanged complexity.

In the construction of Lemma 12, each partial heap at each level of the construction corresponds to all sidetracks with heads taken from some path in the shortest path tree. As each partial heap is formed the corresponding path is formed by

concatenating two shorter paths. We let $pathstart(x)$ for each root of a heap be equal to the endpoint of this path farthest from t . We also store for each partial heap the near endpoint of the path, and the value of the path. Then these values can all be updated in constant time when we merge heaps.

THEOREM 4. *Given a digraph G and a shortest path tree from a vertex s , and given a monoid with values $value(e)$ for each edge $e \in G$, we can compute $value(p)$ for each of the k shortest s - t paths in G , in time and space $O(m + n + k)$.*

5. Dynamic programming applications. Many optimization problems solved by dynamic programming or more complicated matrix searching techniques can be expressed as shortest path problems. Since the graphs arising from dynamic programs are typically acyclic, we can use our algorithm to find longest as well as shortest paths. We demonstrate this approach by a few selected examples.

5.1. The knapsack problem. The *optimization 0-1 knapsack problem* (or *knapsack problem* for short) consists of placing “objects” into a “knapsack” that has room for only a subset of the objects, and maximizing the total value of the included objects. Formally, one is given integers L , c_i , and w_i ($0 \leq i < n$), and one must find $x_i \in \{0, 1\}$ satisfying $\sum x_i c_i \leq L$ and maximizing $\sum x_i w_i$. Dynamic programming solves the problem in time $O(nL)$; Dai et al. [15] show how to find the k best solutions in time $O(knL)$. We now show how to improve this to $O(nL + k)$ using longest paths in a DAG.

Let directed acyclic graph G have $nL + L + 2$ vertices: two terminals s and t and $(n + 1)L$ other vertices with labels (i, j) , $0 \leq i \leq n$ and $0 \leq j \leq L$. Draw an edge from s to each $(0, j)$ and from each (n, j) to t , each having length 0. From each (i, j) with $i < n$, draw two edges: one to $(i + 1, j)$ with length 0, and one to $(i + 1, j + c_i)$ with length w_i (omit this last edge if $j + c_i > L$).

There is a simple one-to-one correspondence between s - t paths and solutions to the knapsack problem: given a path, define x_i to be 1 if the path includes an edge from (i, j) to $(i + 1, j + c_i)$; instead let x_i be 0 if the path includes an edge from (i, j) to $(i + 1, j)$. The length of the path is equal to the corresponding value of $\sum x_i w_i$, so we can find the k best solutions simply by finding the k longest paths in the graph.

THEOREM 5. *We can find the k best solutions to the knapsack problem as defined above, in time $O(nL + k)$.*

5.2. Sequence alignment. The *sequence alignment* or *edit distance* problem is that of matching the characters in one sequence against those of another, obtaining a matching of minimum *cost* where the cost combines terms for mismatched and unmatched characters. This problem and many of its variations can be solved in time $O(xy)$ (where x and y denote the lengths of the two sequences) by a dynamic programming algorithm that takes the form of a shortest path computation in a grid graph.

Byers and Waterman [8, 64] describe a problem of finding all near-optimal solutions to sequence alignment and similar dynamic programming problems. Essentially their problem is that of finding all s - t paths with length less than a given bound L . They describe a simple depth first search algorithm for this problem, which is

especially suited for grid graphs although it will work in any graph and although the authors discuss it in terms of general DAGs. In a general digraph their algorithm would use time $O(k^2m)$ and space $O(km)$. In the acyclic case discussed in the paper, these bounds can be reduced to $O(km)$ and $O(m)$. In grid graphs its performance is even better: time $O(xy + k(x + y))$ and space $O(xy)$. Naor and Brutlag [46] discuss improvements to this technique that among other results include a similar time bound for k shortest paths in grid graphs.

We now discuss the performance of our algorithm for the same length-limited path problem. In general one could apply any k shortest paths algorithm together with a doubling search to find the value of k corresponding to the length limit, but in our case the problem can be solved more simply: simply replace the breadth first search in $H(G)$ with a length-limited depth first search.

THEOREM 6. *We can find the k s - t paths in a graph G that are shorter than a given length limit L , in time $O(m+n+k)$ once a shortest path tree in G is computed.*

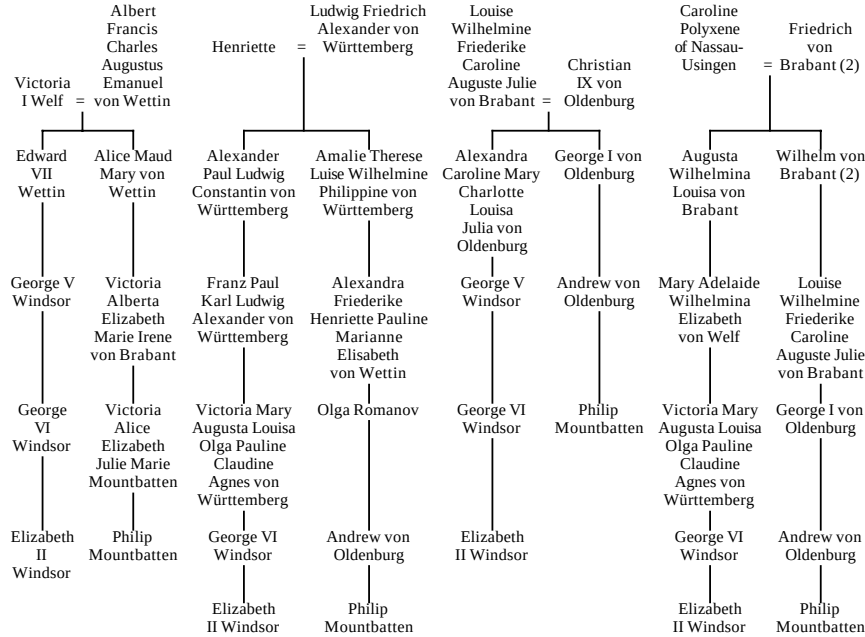
Even for the grid graphs arising in sequence analysis, our $O(xy + k)$ bound improves by a factor of $O(x + y)$ the times of the algorithms of Byers and Waterman [8] and Naor and Brutlag [46].

5.3. Inscribed polygons. We next discuss the problem of, given an n -vertex convex polygon, finding the “best” approximation to it by an r -vertex polygon, $r < n$. This arises, e.g., in computer graphics, in which significant speedups are possible by simplifying the shapes of faraway objects. To our knowledge the “ k best solution” version of the problem has not been studied before. We include it as an example in which the best-known algorithms for the single solution case do not appear to be of the form needed by our techniques; however, one can transform an inefficient algorithm for the original problem into a shortest path problem that with our techniques gives an efficient solution for large enough k .

We formalize the problem as that of finding the maximum area or perimeter convex r -gon inscribed in a convex n -gon. The best known solution takes time $O(n \log n + n\sqrt{r} \log n)$ [1]. However, this algorithm does not appear to be in the form of a shortest path problem, as needed by our techniques.

Instead we describe a less efficient technique for solving the problem by using shortest paths. Number the n -gon vertices v_1, v_2 , etc. Suppose we know that v_i is the lowest numbered vertex to be part of the optimal r -gon. We then form a DAG G_i with $O(rn)$ vertices and $O(rn^2)$ edges, in r levels. In each level we place a copy of each vertex v_j , connected to all vertices with lower numbers in the previous level. Each path from the copy of v_i in the first level of the graph to a vertex in the last level of the graph has r vertices with numbers in ascending order from v_i , and thus corresponds to an inscribed r -gon. We connect one such graph for each initial vertex v_i into one large graph, by adding two vertices s and t , edges from s to each copy of a vertex v_i at the first level of G_i , and edges from each vertex on level r of each G_i to t . Paths in the overall graph G thus correspond to inscribed r -gons with any starting vertex.

It remains to describe the edge lengths in this graph. Edges from s to each v_i will have length zero for either definition of the problem. Edges from a copy of v_i at one level to a copy of v_j at the next level will have length equal to the Euclidean distance from v_i to v_j , for the maximum perimeter version of the problem, and edges connecting a copy of v_j at the last level to t will have length equal to the distance between v_j and the initial vertex v_i . Thus the length of a path becomes exactly the perimeter of the corresponding polygon, and we can find the k best r -gons by finding the k longest paths.

FIG. 7. *Some short relations in a complicated genealogical database.*

For the maximum area problem, we instead let the distance from v_i to v_j be measured by the area of the n -gon cut off by a line segment from v_i to v_j . Thus the total length of a path is equal to the total area outside the corresponding r -gon. Since we want to maximize the area inside the r -gon, we can find the k best r -gons by finding the k shortest paths.

THEOREM 7. *We can find the k maximum area or perimeter r -gons inscribed in an n -gon, in time $O(rn^3 + k)$.*

5.4. Genealogical relations. If one has a database of family relations, one may often wish to determine how some two individuals in the database are related to each other. Formalizing this, one may draw a DAG in which nodes represent people, and an arc connects a parent to each of his or her children. Then each different type of relationship (such as that of being a half-brother, great-aunt, or third cousin twice removed) can be represented as a pair of disjoint paths from a common ancestor (or couple forming a pair of common ancestors) to the two related individuals, with the specific type of relationship being a function of the numbers of edges in each path and of whether the paths begin at a couple or at a single common ancestor. In most families, the DAG one forms in this way has a tree-like structure, and relationships are easy to find. However, in more complicated families with large amounts of intermarriage, one can be quickly overwhelmed with many different relationships. For instance, in the British royal family, Queen Elizabeth and her husband Prince Philip are related in many ways, the closest few being second cousins once removed through King Christian IX of Denmark and his wife Louise, third cousins through Queen Victoria of England and her husband Albert, and fourth cousins through Duke Ludwig Friedrich Alexander of Württemberg and his wife Henriette (Figure 7). The single shortest relationship can be found as a shortest path in a graph formed by combining the DAG with its reversal, but longer paths in this graph

do not necessarily correspond to disjoint pairs of paths. A program my wife, Diana, and I wrote, Gene (<http://www.ics.uci.edu/~eppstein/gene/>), is capable of finding small numbers of relationships quickly using a backtracking search with heuristic pruning, but Gene starts to slow down when asked to produce larger numbers of relationships.

We now describe a technique for applying our k -shortest-path algorithm to this problem, based on a method of Perl and Shiloach [48] for finding shortest pairs of disjoint paths in DAGs. Given a DAG D , we construct a larger DAG D_1 as follows. We first find some topological ordering of D , and let $f(x)$ represent the position of vertex x in this ordering. We then construct one vertex of D_1 for each ordered pair of vertices (x, y) (not necessarily distinct) in D . We also add one additional vertex s in D_1 . We connect (x, y) to (x, z) in D_1 if (y, z) is an arc of D and $f(z) > \max(f(x), f(y))$. Symmetrically, we connect (x, y) to (z, y) if (x, z) is an arc of D and $f(z) > \max(f(x), f(y))$. We connect s to all vertices in D_1 of the form (v, v) .

LEMMA 13. *Let vertices u and v be given. Then the pairs of disjoint paths in D from a common ancestor a to u and v are in one-for-one correspondence with the paths in D_1 from s through (a, a) to (u, v) .*

As a consequence, we can find shortest relationships between two vertices u and v by finding shortest paths in D_1 from s to (u, v) .

THEOREM 8. *Given a DAG with n nodes and m edges, we can construct in $O(mn)$ time a data structure such that, given any two nodes u and v in a DAG, we can list (an implicit representation of) the k shortest pairs of vertex-disjoint paths from a common ancestor to u and v , in time $O(k)$. The same bound holds for listing all pairs with length less than a given bound (where k is the number of such paths). Alternately, the pairs of paths can be output in order by total length in time $O(\log i)$ to list the i th pair. As before, our representation allows constant-time computation of some simple functions of each path, and allows each path to be explicitly generated in time proportional to its length.*

For a proof of Lemma 13 and more details of this application, see [19].

6. Conclusions. We have described algorithms for the k -shortest-paths problem, improving by an order of magnitude previously known bounds. The asymptotic performance of the algorithm makes it an especially promising choice in situations when large numbers of paths are to be generated, and there already exist at least two implementations: one by Shibuya, Imai, and coworkers [52, 53, 54, 55] and one by Martins (<http://www.mat.uc.pt/~eqvm/eqvm.html>).

We list the following as open problems.

- The linear time construction when the shortest path tree is known is rather complicated. Is there a simpler method for achieving the same result? How quickly can we maintain heaps $H_T(v)$ if new leaves are added to the tree? (Lemma 4 solves this in $O(\log n)$ time per vertex, but it seems that at least $O(\log \log n)$ should be possible.)
- As described above, we can find the k best inscribed r -gons in an n -gon, in time $O(rn^3 + k)$. However, the best single-optimum solution has the much faster time bound $O(n \log n + n\sqrt{r \log n})$ [1]. Our algorithms for the k best r -gons are efficient (in the sense that we use constant time per r -gon) only when $k = \Omega(rn^3)$. The same phenomenon of overly large preprocessing times also occurs in our application to genealogical relationship finding: the shortest relationship can be found in linear time, but our k -shortest-relationship method takes time $O(mn + k)$. Can we improve these bounds?

- Are there properties of paths not described by monoids which we can nevertheless compute efficiently from our representation? In particular, how quickly can we test whether each path generated is simple?

Acknowledgments. I thank Greg Frederickson, Sandy Irani, and George Lueker for helpful comments on drafts of this paper.

REFERENCES

- [1] A. AGGARWAL, B. SCHIEBER, AND T. TOKUYAMA, *Finding a minimum weight K -link path in graphs with Monge property and applications*, in Proc. 9th Symp. Computational Geometry, Assoc. for Computing Machinery, 1993, pp. 189–197.
- [2] R. K. AHUJA, K. MEHLHORN, J. B. ORLIN, AND R. E. TARJAN, *Faster algorithms for the shortest path problem*, J. Assoc. Comput. Mach., 37 (1990), pp. 213–223.
- [3] J. A. AZEVEDO, M. E. O. SANTOS COSTA, J. J. E. R. SILVESTRE MADEIRA, AND E. Q. V. MARTINS, *An algorithm for the ranking of shortest paths*, Eur. J. Operational Research, 69 (1993), pp. 97–106.
- [4] A. BAKO, *All paths in an activity network*, Mathematische Operationsforschung und Statistik, 7 (1976), pp. 851–858.
- [5] A. BAKO AND P. KAS, *Determining the k -th shortest path by matrix method*, Szigma, 10 (1977), pp. 61–66 (in Hungarian).
- [6] R. E. BELLMAN, *On a routing problem*, Quart. Appl. Math., 16 (1958), pp. 87–90.
- [7] A. W. BRANDER AND M. C. SINCLAIR, *A comparative study of k -shortest path algorithms*, in Proc. 11th UK Performance Engineering Workshop for Computer and Telecommunications Systems, 1995.
- [8] T. H. BYERS AND M. S. WATERMAN, *Determining all optimal and near-optimal solutions when solving shortest path problems by dynamic programming*, Oper. Res., 32 (1984), pp. 1381–1384.
- [9] P. CARRARESI AND C. SODINI, *A binary enumeration tree to find K shortest paths*, in Proc. 7th Symp. Operations Research, Methods Oper. Res. 45, Athenäum/Hain/Hanstein, 1983, pp. 177–188.
- [10] G.-H. CHEN AND Y.-C. HUNG, *Algorithms for the constrained quickest path problem and the enumeration of quickest paths*, Comput. Oper. Res., 21 (1994), pp. 113–118.
- [11] Y. L. CHEN, *An algorithm for finding the k quickest paths in a network*, Comput. Oper. Res., 20 (1993), pp. 59–65.
- [12] Y. L. CHEN, *Finding the k quickest simple paths in a network*, Inform. Process. Lett., 50 (1994), pp. 89–92.
- [13] E. I. CHONG, S. R. MADDILA, AND S. T. MORLEY, *On finding single-source single-destination k shortest paths*, in Proc. 7th Int. Conf. Computing and Information, Trent University, Canada, 1995.
- [14] A. CONSIGLIO AND A. PECORELLA, *Using simulated annealing to solve the K -shortest path problem*, in Proc. Conf. Italian Assoc. Operations Research, 1995.
- [15] Y. DAI, H. IMAI, K. IWANO, AND N. KATOH, *How to treat delete requests in semi-online problems*, in Proc. 4th Int. Symp. Algorithms and Computation, Lecture Notes in Comput. Sci. 762, Springer-Verlag, New York, 1993, pp. 48–57.
- [16] M. T. DICKERSON AND D. EPPSTEIN, *Algorithms for proximity problems in higher dimensions*, Comput. Geom., 5 (1996), pp. 277–291.
- [17] S. E. DREYFUS, *An appraisal of some shortest path algorithms*, Oper. Res., 17 (1969), pp. 395–412.
- [18] EL-AMIN AND AL-GHAMDI, *An expert system for transmission line route selection*, in Int. Power Engineering Conf, Vol. 2, Nanyang Technol. Univ., Singapore, 1993, pp. 697–702.
- [19] D. EPPSTEIN, *Finding common ancestors and disjoint paths in DAGs*, Tech. Report 95-52, Univ. of California, Dept. Information and Computer Science, Irvine, 1995.
- [20] D. EPPSTEIN, *Ten algorithms for Egyptian fractions*, Mathematica in Education and Research, 4 (1995), pp. 5–15.
- [21] D. EPPSTEIN, Z. GALIL, AND G. F. ITALIANO, *Improved sparsification*, Tech. Report 93-20, Univ. of California, Dept. Information and Computer Science, Irvine, 1993.
- [22] D. EPPSTEIN, Z. GALIL, G. F. ITALIANO, AND A. NISSENZWEIG, *Sparsification—A technique for speeding up dynamic graph algorithms*, in Proc. 33rd Symp. Foundations of Computer Science, IEEE, 1992, pp. 60–69.
- [23] L. R. FORD, JR. AND D. R. FULKERSON, *Flows in Networks*, Princeton University Press, Prince-

- ton, NJ, 1962.
- [24] B. L. FOX, *k-th shortest paths and applications to the probabilistic networks*, in ORSA/TIMS Joint National Mtg., Vol. 23, 1975, p. B263.
 - [25] G. N. FREDERICKSON, *Ambivalent data structures for dynamic 2-edge-connectivity and k smallest spanning trees*, in Proc. 32nd Symp. Foundations of Computer Science, IEEE, 1991, pp. 632–641.
 - [26] G. N. FREDERICKSON, *An optimal algorithm for selection in a min-heap*, Inform. and Comput., 104 (1993), pp. 197–214.
 - [27] M. L. FREDMAN AND R. E. TARJAN, *Fibonacci heaps and their uses in improved network optimization algorithms*, J. Assoc. Comput. Mach., 34 (1987), pp. 596–615.
 - [28] M. L. FREDMAN AND D. E. WILLARD, *Trans-dichotomous algorithms for minimum spanning trees and shortest paths*, in Proc. 31st Symp. Foundations of Computer Science, IEEE, 1990, pp. 719–725.
 - [29] A. V. GOLDBERG, *Scaling algorithms for the shortest paths problem*, SIAM J. Comput., 24 (1995), pp. 494–504.
 - [30] V. HATZIVASSILOGLU AND K. KNIGHT, *Unification-based glossing*, in Proc. 14th Int. Joint Conf. Artificial Intelligence, 1995, Morgan Kaufmann, San Francisco, pp. 1382–1389.
 - [31] G. J. HORNE, *Finding the K least cost paths in an acyclic activity network*, J. Oper. Res. Soc., 31 (1980), pp. 443–448.
 - [32] L.-M. JIN AND S.-P. CHAN, *An electrical method for finding suboptimal routes*, in Int. Symp. Circuits and Systems, Vol. 2, IEEE, 1989, pp. 935–938.
 - [33] D. B. JOHNSON, *A priority queue in which initialization and queue operations take $O(\log \log D)$ time*, Math. Systems Theory, 15 (1982), pp. 295–309.
 - [34] N. KATOH, T. IBARAKI, AND H. MINE, *An $O(Kn^2)$ algorithm for K shortest simple paths in an undirected graph with nonnegative arc length*, Trans. Inst. Electronics and Communication Engineers of Japan, E61 (1978), pp. 971–972.
 - [35] N. KATOH, T. IBARAKI, AND H. MINE, *An efficient algorithm for K shortest simple paths*, Networks, 12 (1982), pp. 411–427.
 - [36] P. N. KLEIN, S. RAO, M. H. RAUCH, AND S. SUBRAMANIAN, *Faster shortest-path algorithms for planar graphs*, in Proc. 26th Symp. Theory of Computing, Assoc. for Computing Machinery, 1994, pp. 27–37.
 - [37] N. KUMAR AND R. K. GHOSH, *Parallel algorithm for finding first K shortest paths*, Computer Science and Informatics, 24 (1994), pp. 21–28.
 - [38] A. G. LAW AND A. REZAZADEH, *Computing the K-shortest paths, under nonnegative weighting*, in Proc. 22nd Manitoba Conf. Numerical Mathematics and Computing, Congr. Numer., 92 (1993), pp. 277–280.
 - [39] E. L. LAWLER, *A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem*, Management Science, 18 (1972), pp. 401–405.
 - [40] E. L. LAWLER, *Comment on computing the k shortest paths in a graph*, Commun. Assoc. Comput. Mach., 20 (1977), pp. 603–604.
 - [41] E. Q. V. MARTINS, *An algorithm for ranking paths that may contain cycles*, Eur. J. Oper. Res., 18 (1984), pp. 123–130.
 - [42] S.-P. MIAOU AND S.-M. CHIN, *Computing k-shortest path for nuclear spent fuel highway transportation*, Eur. J. Oper. Res., 53 (1991), pp. 64–80.
 - [43] E. MINIEKA, *On computing sets of shortest paths in a graph*, Commun. Assoc. Comput. Mach., 17 (1974), pp. 351–353.
 - [44] E. MINIEKA, *The K-th shortest path problem*, in ORSA/TIMS Joint National Mtg., Vol. 23, 1975, p. B/116.
 - [45] E. MINIEKA AND D. R. SHIER, *A note on an algebra for the k best routes in a network*, J. Inst. Mathematics and Its Applications, 11 (1973), pp. 145–149.
 - [46] D. NAOR AND D. BRUTLAG, *On near-optimal alignments of biological sequences*, J. Computational Biology, 1 (1994), pp. 349–366.
 - [47] A. PERKO, *Implementation of algorithms for K shortest loopless paths*, Networks, 16 (1986), pp. 149–160.
 - [48] Y. PERL AND Y. SHILOACH, *Finding two disjoint paths between two pairs of vertices in a graph*, J. Assoc. Comput. Mach., 25 (1978), pp. 1–9.
 - [49] J. B. ROSEN, S.-Z. SUN, AND G.-L. XUE, *Algorithms for the quickest path problem and the enumeration of quickest paths*, Comput. Oper. Res., 18 (1991), pp. 579–584.
 - [50] E. RUPPERT, *Finding the k shortest paths in parallel*, in Proc. 14th Symp. Theoretical Aspects of Computer Science, Lecture Notes in Comput. Sci. 1200, Springer-Verlag, New York, 1997, pp. 475–486.

- [51] T. SHIBUYA, *Finding the k shortest paths by AI search techniques*, Cooperative Research Reports in Modeling and Algorithms, 7 (1995), pp. 212–222.
- [52] T. SHIBUYA, T. IKEDA, H. IMAI, S. NISHIMURA, H. SHIMOURA, AND K. TENMOKU, *Finding a realistic detour by AI search techniques*, in Proc. 2nd Intelligent Transportation Systems, 4 (1995), pp. 2037–2044.
- [53] T. SHIBUYA AND H. IMAI, *Enumerating suboptimal alignments of multiple biological sequences efficiently*, in Proc. 2nd Pacific Symp. Biocomputing, World Scientific, Singapore, 1997, pp. 409–420.
- [54] T. SHIBUYA AND H. IMAI, *New flexible approaches for multiple sequence alignment*, in Proc. 1st Int. Conf. Computational Molecular Biology, Assoc. for Computing Machinery, 1997, pp. 267–276.
- [55] T. SHIBUYA, H. IMAI, S. NISHIMURA, H. SHIMOURA, AND K. TENMOKU, *Detour queries in geographical databases for navigation and related algorithm animations*, in Proc. Int. Symp. Cooperative Database Systems for Advanced Applications, 2 (1996), pp. 333–340.
- [56] D. R. SHIER, *Algorithms for finding the k shortest paths in a network*, in ORSA/TIMS Joint National Mtg., 1976, p. 115.
- [57] D. R. SHIER, *Iterative methods for determining the k shortest paths in a network*, Networks, 6 (1976), pp. 205–229.
- [58] D. R. SHIER, *On algorithms for finding the k shortest paths in a network*, Networks, 9 (1979), pp. 195–214.
- [59] C. C. SKICISM AND B. L. GOLDEN, *Solving k -shortest and constrained shortest path problems efficiently*, in Network Optimization and Applications, B. Shetty, ed., Annals Oper. Res. 20, Baltzer Science Publishers, Bussum, The Netherlands, 1989, pp. 249–282.
- [60] K. SUGIMOTO AND N. KATOH, *An algorithm for finding k shortest loopless paths in a directed network*, Trans. Information Processing Soc. Japan, 26 (1985), pp. 356–364 (in Japanese).
- [61] J. W. SUURBALLE, *Disjoint paths in a network*, Networks, 4 (1974), pp. 125–145.
- [62] R. E. TARJAN, *Data Structures and Network Algorithms*, CBMS-NSF Reg. Conf. Ser. Appl. Math. 44, SIAM, Philadelphia, 1983.
- [63] R. THUMER, *A method for selecting the shortest path of a network*, Z. Oper. Res., Serie B (Praxis), 19 (1975), pp. B149–153 (in German).
- [64] M. S. WATERMAN, *Sequence alignments in the neighborhood of the optimum*, Proc. Natl. Acad. Sci. USA, 80 (1983), pp. 3123–3124.
- [65] M. M. WEIGAND, *A new algorithm for the solution of the k -th best route problem*, Computing, 16 (1976), pp. 139–151.
- [66] A. WONGSEELASHOTE, *An algebra for determining all path-values in a network with application to k -shortest-paths problems*, Networks, 6 (1976), pp. 307–334.
- [67] A. WONGSEELASHOTE, *Semirings and path spaces*, Discrete Math., 26 (1979), pp. 55–78.
- [68] J. Y. YEN, *Finding the K shortest loopless paths in a network*, Management Science, 17 (1971), pp. 712–716.
- [69] J. Y. YEN, *Another algorithm for finding the K shortest-loopless network paths*, in Proc. 41st Mtg. Operations Research Society of America, 20 (1972), p. B/185.